



RoyalRent: resumen para defender el proyecto

Documento corto para explicar la idea, la arquitectura, las funcionalidades clave y las preguntas que pueden haceros en la defensa del 2 de junio de 2026.

Idea central

Plataforma web para reservar coches de lujo, yates y helicópteros desde un catálogo único.

Enfoque DAM

Frontend, backend, base de datos, autenticación, API REST y gestión de estado.

Uso del PDF

No lo memorices entero: úsalo como chuleta de orden y respuestas clave.

1. MENSAJE PRINCIPAL

Que es RoyalRent

RoyalRent es una aplicacion web full-stack que centraliza el alquiler de servicios premium: coches de lujo, yates y helicopteros. El usuario puede explorar el catalogo, filtrar por categoria, abrir una ficha con galeria y calendario, anadir servicios al carrito, simular el pago, registrar su cuenta y consultar su historial de reservas.

Frase de apertura recomendada: "Nuestro proyecto se llama RoyalRent y nace como una plataforma de alquiler premium. Hemos querido demostrar una aplicacion completa, no solo una pagina visual: tiene frontend, API, base de datos, autenticacion, reservas, perfil de usuario y panel de administracion."

Problema que resuelve

- Evita tener servicios premium dispersos en varias paginas.
- Permite consultar productos, precios, fechas y reservas desde un mismo sitio.
- Ofrece una experiencia visual acorde al sector: oscura, elegante y con detalles dorados.

Objetivo academico

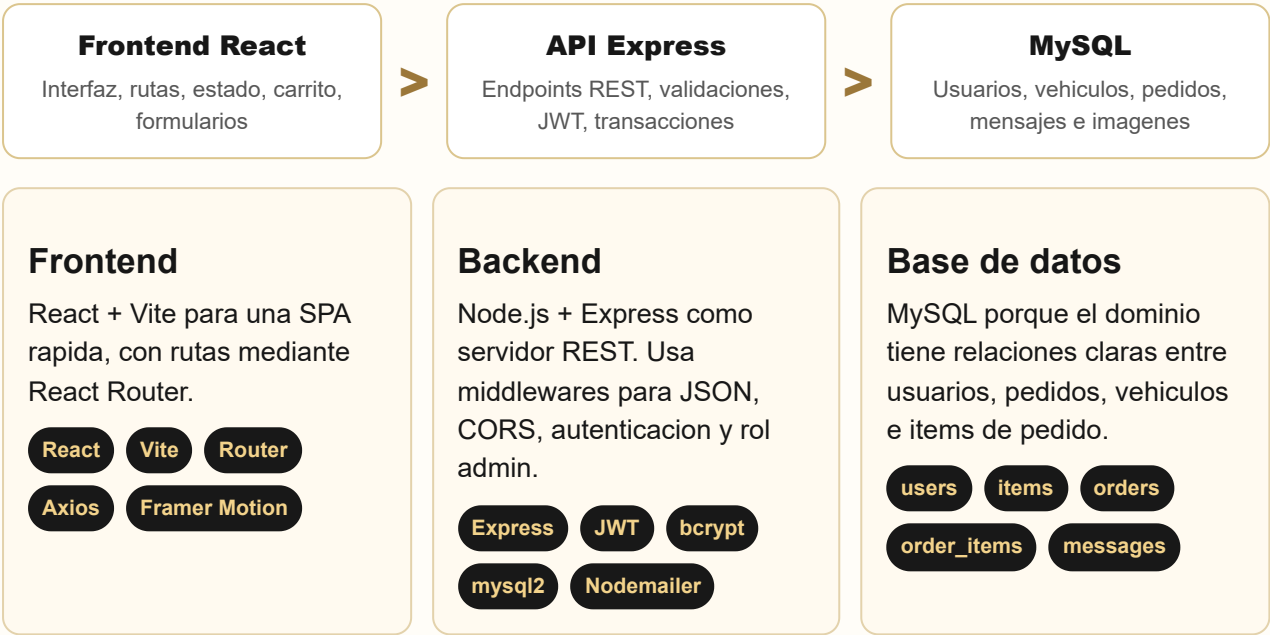
- Aplicar programacion web moderna con React.
- Crear una API REST con Node.js y Express.
- Modelar una base de datos relacional en MySQL.
- Integrar autenticacion, roles, carrito y persistencia.

Funcionalidades que conviene enseñar en la demo

- 1 Inicio:** explicar marca, video hero, servicios, equipo y formulario de contacto.
- 2 Catalogo:** filtrar por coches, yates y helicopteros; destacar que los datos vienen de la API.
- 3 Ficha de servicio:** galeria, descripcion, precio, calendario y validacion de fechas.
- 4 Carrito y pago:** resumen, calculo por dias, pago simulado y creacion de pedido.
- 5 Perfil:** datos del usuario, actualizacion de cuenta e historial de reservas.
- 6 Admin:** estadisticas, usuarios, pedidos, vehiculos, disponibilidad y mensajes.

2. ARQUITECTURA

Como esta construido



GET /api/orders/history	Historial del usuario.	Solo se accede con token.
/api/admin/*	Estadísticas, usuarios, pedidos, items y mensajes.	Solo usuarios con rol admin .
POST /api/contact	Formulario de contacto.	Guarda mensajes y puede enviar correo si hay SMTP configurado.

3. BASE DE DATOS Y SEGURIDAD

Lo tecnico que mas puntua

Modelo de datos

- `users` : datos de usuario, email unico, password hasheada y rol.
- `items` : vehiculos con tipo, descripcion, precio, imagen y disponibilidad general.
- `item_images` : galeria de imagenes por vehiculo.
- `orders` : pedido asociado a un usuario y total.
- `order_items` : detalle de cada vehiculo reservado, cantidad, precio y fechas.
- `messages` : mensajes enviados desde contacto.

Relaciones

- Un usuario puede tener muchos pedidos.
- Un pedido puede contener varios vehiculos.
- Cada detalle de pedido une `orders` con `items`.
- Cada vehiculo puede tener varias imagenes.
- Los mensajes se guardan de forma independiente para administracion.

Seguridad explicada de forma clara

MEDIDA	COMO SE APLICA	COMO DECIRLO EN DEFENSA
JWT	El login devuelve un token y el backend lo verifica en rutas protegidas.	"No cualquiera puede ver perfil, carrito finalizado, historial o admin."
bcrypt	La password no se guarda en claro, se compara contra un hash.	"Si alguien viera la base de datos, no podria leer las contraseñas directamente."
Roles	El middleware admin comprueba <code>user.role === 'admin'</code> .	"Separarnos usuario normal y administrador."
Consultas parametrizadas	Se usan placeholders <code>?</code> con <code>mysql2</code> .	"Reducimos riesgo de inyeccion SQL."
Transacciones	Crear pedido e insertar sus items va dentro de una transaccion.	"Si algo falla, se revierte todo y no queda un pedido a medias."
Variables de entorno	Configuracion de BD, JWT y correo mediante <code>.env</code> .	"La configuracion sensible no deberia ir fija en el codigo."

Punto honesto si os preguntan por limites: el pago es simulado y la disponibilidad actual es general por vehiculo. Una mejora real seria integrar Stripe/PayPal y comprobar disponibilidad por rangos de fechas para impedir solapamientos exactos.

Credencial de demo

El script de base de datos crea un administrador de prueba: `admin@royalrent.com` con contraseña `admin123` . Usadlo solo como usuario local de demostracion.

4. GUION ORAL

Defensa sugerida en 10-12 minutos

- 1 Presentacion - 1 min:** "Somos Gonzalo y Miguel Jose, y presentamos RoyalRent, una plataforma full-stack para alquiler premium."
- 2 Motivacion - 1 min:** explicar que el objetivo era crear una experiencia completa: catalogo, usuario, reserva y administracion.
- 3 Tecnologias - 2 min:** React/Vite en frontend, Express en backend, MySQL para datos relacionales, JWT y bcrypt para autenticacion.
- 4 Demo usuario - 4 min:** Home, catalogo, filtro, ficha, calendario, carrito, pago simulado, perfil e historial.
- 5 Demo admin - 2 min:** entrar como admin y enseñar estadísticas, usuarios, pedidos, vehículos, disponibilidad y mensajes.
- 6 Decisiones y aprendizaje - 1 min:** separar frontend/backend, usar API REST, transacciones y contexto de React.
- 7 Mejoras futuras - 1 min:** pasarela de pago real, despliegue cloud, tests, disponibilidad por fechas, buscador avanzado.

Que puede decir quien lleve mas peso

- Arquitectura completa: React, API Express y MySQL.
- Flujo de datos: el frontend llama a endpoints y la API consulta la BD.
- Seguridad: JWT, bcrypt, roles y rutas protegidas.
- Transacciones al crear pedidos.
- Limitaciones y mejoras futuras.

Que puede decir tu amigo si tiene menos tiempo

- Presentar la idea y el problema.
- Explicar la experiencia de usuario: home, catalogo, ficha y carrito.
- Defender la parte visual: marca premium, colores oscuros y dorados.
- Cerrar con aprendizaje y mejoras futuras.
- Aprender las 8 frases de la pagina siguiente.

Frases comodin para sonar seguros

- "Hemos separado responsabilidades: React se encarga de la interfaz y Express de la logica de servidor."
- "La base de datos es relacional porque hay relaciones claras entre usuarios, pedidos e items."
- "El carrito se mantiene en el navegador para no perder la seleccion al recargar."
- "Las rutas privadas verifican que el usuario este autenticado antes de continuar."

- "El panel admin demuestra que la aplicacion no solo es de cliente, tambien permite gestion interna."

5. CHULETA EXPRESS

Version rapida para tu amigo

Si no le da tiempo a estudiar todo, que se aprenda esto. Con estas ideas puede defender una parte digna sin meterse en detalles que no controle.

Resumen en 30 segundos

"RoyalRent es una web full-stack para alquilar coches de lujo, yates y helicopteros. Tiene una parte publica con catalogo y contacto, una parte de usuario con registro, carrito, pago simulado e historial, y una parte de administrador para gestionar usuarios, pedidos, vehiculos y mensajes. La hemos construido con React, Node/Express y MySQL."

8 cosas que debe saber si o si

1. Es una aplicacion completa, no una pagina estatica.
2. El catalogo viene de una base de datos MySQL.
3. El usuario puede registrarse, iniciar sesion y reservar.
4. El carrito calcula el precio segun fechas y cantidad.
5. El pago esta simulado para el proyecto academico.
6. El perfil muestra el historial real de reservas.
7. El administrador tiene un panel propio con datos de gestion.
8. La seguridad usa JWT para sesion y bcrypt para contraseñas.

Su mini guion

1. "La idea es centralizar servicios premium en una sola plataforma."
2. "La experiencia visual busca transmitir lujo: fondo oscuro, dorado, imagenes grandes y animaciones suaves."
3. "En servicios se puede filtrar por categoria y abrir cada vehiculo para ver detalles."
4. "El usuario elige fechas, lo anade al carrito y finaliza una reserva simulada."
5. "Como mejora futura, meteriamos pasarela de pago real y disponibilidad por fechas."

Si le preguntan algo tecnico y no lo sabe

Respuesta segura: "Esa parte la trabajo mas mi companero, pero a nivel general funciona asi: el frontend hace una peticion a la API, la API valida el usuario o los datos necesarios y despues consulta o actualiza MySQL. Lo importante es que las capas estan separadas."

6. PREGUNTAS PROBABLES

Respuestas preparadas

PREGUNTA	RESPUESTA CORTA
Por que React?	Porque permite construir una SPA por componentes, reutilizar partes como Navbar, Drawer o Context, y actualizar la interfaz sin recargar toda la pagina.
Por que Express?	Porque es ligero, directo y encaja muy bien para crear una API REST en Node.js, suficiente para el alcance del proyecto.
Por que MySQL?	Porque los datos son relacionales: usuarios, pedidos, vehiculos e items de pedido. Ademas encaja con lo aprendido en DAM.
Como se protege la sesion?	Con JWT. Al iniciar sesion se guarda un token y el backend lo verifica en rutas protegidas.
Como se protegen las contraseñas?	No se guardan en claro. Se hashean con bcrypt antes de insertarlas en la base de datos.
El pago es real?	No. Es un flujo simulado para demostrar checkout y validacion de formulario. En produccion se integraria una pasarela real.
Que pasa si falla un pedido?	El backend usa transacciones: si falla una parte, se hace rollback y no queda un pedido incompleto.
Que aporta el panel admin?	Permite gestionar la aplicacion desde dentro: usuarios, pedidos, vehiculos, mensajes y disponibilidad.
Que mejorariais?	Pago real, despliegue en nube, tests automatizados, busqueda avanzada, control de disponibilidad por fechas y cookies HttpOnly para el token.
Que habeis aprendido?	A conectar frontend, backend y base de datos en una aplicacion completa, manejar autenticacion, roles, estado global y operaciones CRUD.

Consejo para la defensa: no intentéis recitar tecnologia por tecnologia. Enseñad un flujo real de usuario y, cuando salga cada parte, explicad que tecnologia interviene. Eso suena mas natural y demuestra que entendéis el proyecto.

Plan B si la demo falla

- Tened abierta la memoria grande `Documentacion_TFG.pdf`.
- Explicad el flujo con este orden: React -> API -> MySQL.
- Mostrad el codigo de `server.js` para endpoints y `schema.sql` para la base de datos.
- Decid con calma: "La demo depende del entorno local, pero la arquitectura y el codigo estan preparados."